

Personal Reflection Report

Zhicheng Zhang 2023115862

1. Role and Scope

At the start of the project, my primary Belbin role was the **Shaper (SH)**. In practice, I worked as a **frontend engineer** and a **product manager**. My concrete responsibility was to deliver the **user app** and the **dispatcher app**, and I learned **uni-app** quickly so I could work across both frontend codebases.

The Shaper role fit my working style well. I tended to push the team to turn abstract ideas into concrete deliverables and to keep the project moving. In this project, that meant translating requirements into pages, confirming interfaces with the backend team, and making sure the frontend work stayed aligned with the overall product direction.

The first example was the early requirement and prototype stage. I helped clarify the workflows for scooter discovery, unlock and lock operations, ride tracking, and map-based management. I also participated in the wireframe discussion so the page structure was settled before implementation. This showed the Shaper role because I was actively driving the team from vague ideas to an executable plan.

The second example was the actual frontend delivery work. I implemented the rider-side pages and the dispatcher-side pages, and I used my prior frontend experience to learn uni-app quickly. This allowed me to switch between the two apps without slowing the team down. It also showed that I could take responsibility for real deliverables rather than only discuss requirements.

2. Key Contributions

2.1 Requirement Understanding and Prototype Design

I clarified requirements and page flows for three user groups (scooter discovery, unlock/lock, ride tracking, map management), reducing ambiguity and rework.

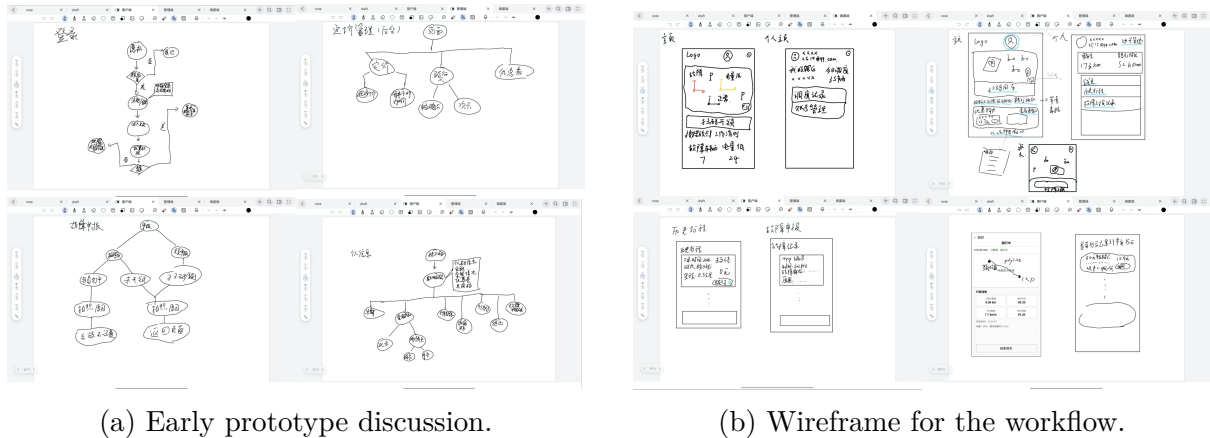


Figure 1: Prototype discussion and wireframe used to define the main user flow.

2.2 API Collaboration

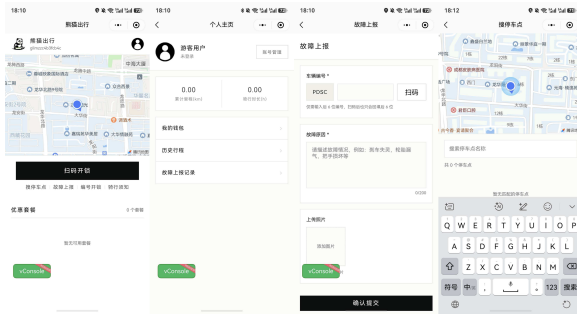
To keep frontend and backend integration efficient, I used Apifox to align interface definitions and test the API before implementation.



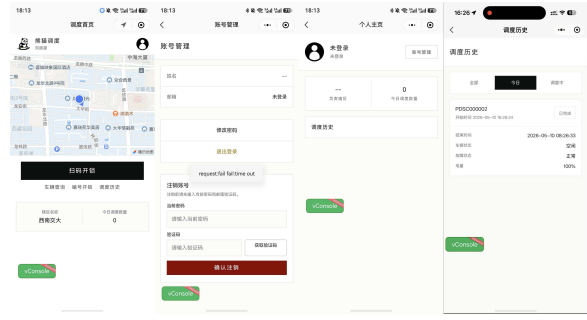
Figure 2: API coordination in Apifox.

2.3 Frontend Delivery

I translated the agreed design into the actual rider and dispatcher pages, making sure the two apps matched the intended workflow.



(a) User-side pages.



(b) Dispatcher-side pages.

Figure 3: The two frontend applications I delivered as the frontend engineer.

3. Strengths

Two strengths stood out in my role: **communication and adaptability**. Because I understood both product goals and implementation details, I could explain user needs clearly to the team and turn technical constraints back into practical interface decisions.

The first part of this strength was my ability to handle early-stage design thinking. Before implementation, I helped the team discuss the user flow and the wireframe, so the interface direction was clear before code writing started. This improved team alignment and reduced repeated changes.

Another strength was **frontend framework experience and fast learning**. Since I had experience on frontend framework like Vue and React, I learned **uni-app** quickly and became productive across both frontend codebases in a short time. That helped the team keep momentum during implementation because I could contribute to two separate frontend streams without a long ramp-up period.

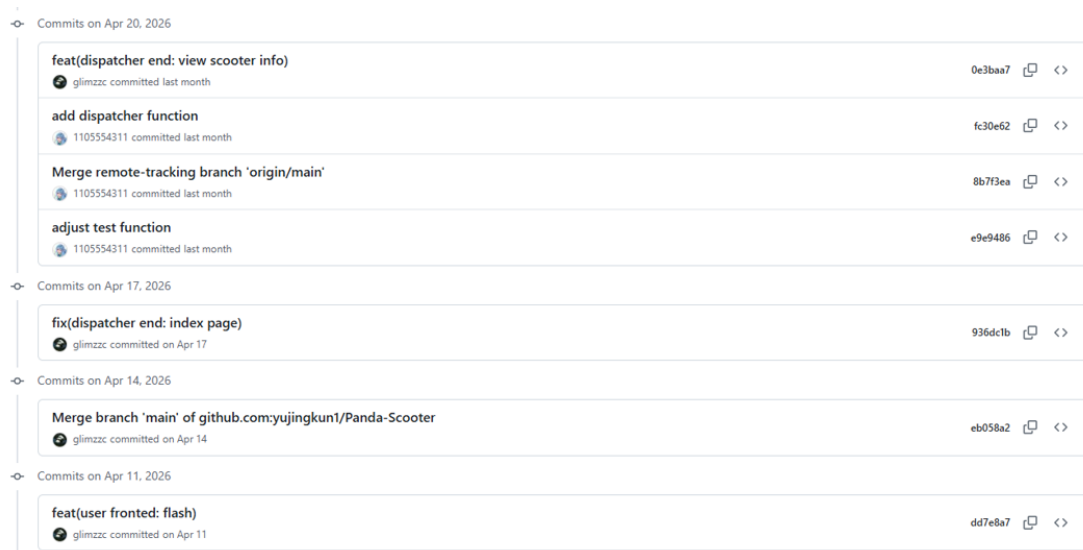


Figure 4: GitHub commit history showing that I ramped up quickly and delivered iteratively across tasks.

4. Weaknesses and Improvement

My first weakness was that, as a frontend engineer, my UI design ability was not as polished as a dedicated designer's. The team had already produced wireframes, but I still had to make many concrete UI decisions during implementation. That meant some visual design work was left to me at the coding stage, and the division of work was not ideal. To deal with this, I used the wireframes as a baseline and kept the UI simple and consistent instead of trying to redesign the whole interface.

My second weakness was that my task load was too broad. I had to complete coding work while also participating in increasing requirement implementation planning, so I could not focus on one area completely. This sometimes split my attention between design, development, and coordination, which affected efficiency and made me switch context more often than I should have. I dealt with this by using short communication cycles, confirming requirements before coding, and breaking the work into smaller steps.

Overall, these weaknesses showed that I was effective in delivery, but I still needed better balance between visual design, implementation, and planning.

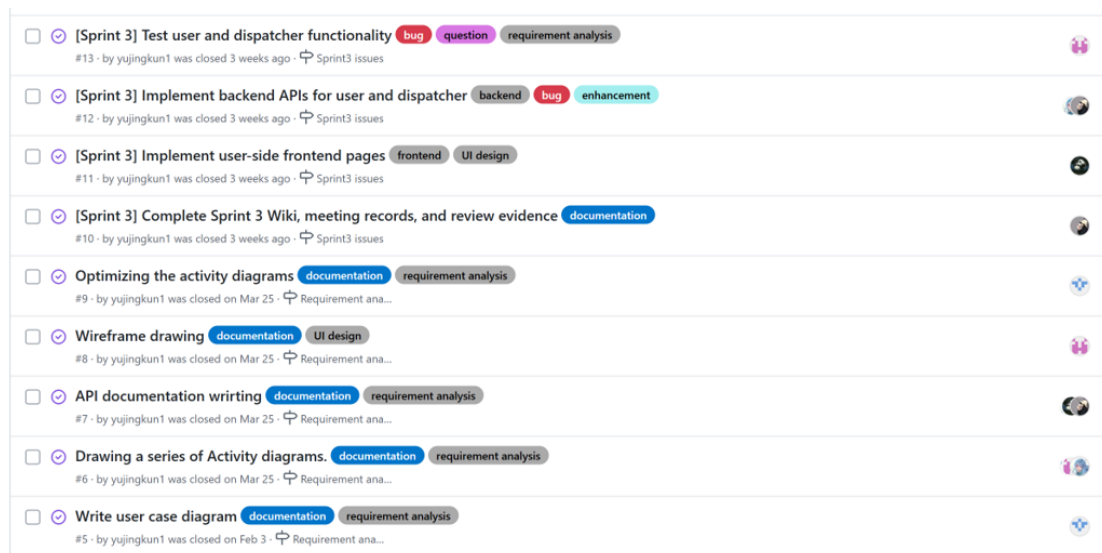


Figure 5: GitHub wiki (Tasks involve requirement analysis, API document writing and pages implementation).